

POSTGRESQL

Backup & Recovery Tools

Native Utilities • pgBackRest • Barman • WAL-G • Cloud & Enterprise Recovery

Target audience	Senior PostgreSQL DBA / Cloud DBA / Database Engineer / Architect
Primary version baseline	PostgreSQL 18 documentation
Validation date	9 September 2026
Purpose	Enterprise backup design, operations, PITR, restore testing, and interview preparation

Enterprise takeaway: For a self-managed production PostgreSQL estate, pgBackRest is the strongest general-purpose enterprise choice in this comparison. Barman is especially strong when centralized management of many PostgreSQL servers is a primary requirement. Native PostgreSQL backup tools remain mandatory DBA knowledge and are the foundation that third-party platforms build upon.

Technical SOP • Operations Runbook • Decision Guide

Contents

1. Scope and Backup Strategy Principles
2. PostgreSQL Backup Methods: Logical vs Physical vs Continuous Archiving
3. Native Tool: `pg_dump`
4. Native Tool: `pg_restore`
5. Native Tool: `pg_dumpall` and `psql`
6. Native Tool: `pg_basebackup`
7. Native Tool: Native Incremental Backup and `pg_combinebackup`
8. Native Tool: `pg_verifybackup`
9. Native WAL Archiving and Point-in-Time Recovery (PITR)
10. Native Low-Level Backup API
11. Enterprise Tool: `pgBackRest`
12. Enterprise Tool: Barman
13. Cloud-Focused Tool: WAL-G
14. Other Third-Party Tools: `pg_probackup`, `pghoard`, WAL-E, `pg_rman`
15. Storage Snapshots and Cloud-Managed Backups
16. Enterprise Comparison Matrix
17. Architecture Patterns and Tool Selection
18. RPO/RTO Design Guidance
19. Backup Scheduling and Retention Examples
20. Monitoring, Validation and Restore Testing
21. Real-Time Failure and Recovery Scenarios
22. Security and Compliance Controls
23. Production Runbook Checklists
24. Interview / Scenario Questions
25. Final Recommendations
26. Official References

1. Scope and Backup Strategy Principles

A PostgreSQL backup strategy is not just a command or a backup file. An enterprise design must prove that data can be restored within the required recovery time, to an acceptable recovery point, under real failure conditions.

- Use more than one recovery method where the business risk requires it. Logical export, physical backup, WAL archiving and infrastructure snapshots solve different problems.
- Define RPO (maximum acceptable data loss) and RTO (maximum acceptable recovery duration) before selecting backup frequency and storage.
- Keep backup data outside the database server failure domain. A backup on the same disk as PGDATA is not a disaster-recovery copy.
- Protect the WAL archive. PITR cannot reach a target point when required WAL segments are missing.
- Automate retention, but never delete a base backup or WAL that is still needed by a dependent incremental chain or recovery window.
- Verify backups and perform scheduled restore tests. A successful backup job is not the same as a successful recovery.
- Document credentials, encryption keys, object-storage permissions, network dependencies and operational ownership.

PostgreSQL baseline: PostgreSQL documents three fundamental approaches: SQL dump, file-system-level backup, and continuous archiving. Enterprise products automate and extend these foundations.

2. PostgreSQL Backup Methods

Method	Examples	Granularity	PITR	Best use
Logical	pg_dump, pg_dumpall	Database / schema / table / object	No, not by itself	Migrations, object-level recovery, cross-version export
Physical base backup	pg_basebackup, pgBackRest, Barman, WAL-G	Whole PostgreSQL cluster	Yes, with WAL	DR, fast full-cluster recovery, standby creation
Continuous archiving	archive_command/archive_library + WAL	WAL stream	Yes	Recover to a selected time/LSN/XID
Storage snapshot	LVM/ZFS/SAN/cloud volume snapshot	Volume / disk	Possible when coordinated with WAL	Very fast snapshot/restore at infrastructure layer
Managed-service backup	RDS/Aurora/Azure/GCP facilities	Managed DB instance/service	Provider dependent, normally yes	Managed PostgreSQL services

3. Native Tool: pg_dump

pg_dump creates a consistent logical export of one PostgreSQL database while normal database activity continues.

- Exports one database at a time.
- Supports plain SQL, custom, tar and directory archive formats.
- Custom and directory formats work with pg_restore and allow selective restore.

- Directory format supports parallel dump jobs.
- Useful for major-version migration because logical dumps are more portable than physical cluster files.
- Not normally the best single solution for routine backup of very large production clusters.

Common commands

```
# Custom-format logical backup
pg_dump -h pgsql01 -U backupuser -F c -f /backup/dvdrental.dump dvdrental

# Directory-format parallel backup
pg_dump -h pgsql01 -U backupuser -F d -j 8 -f /backup/dvdrental_dir dvdrental

# Schema-only
pg_dump -h pgsql01 -U backupuser --schema-only -F c -f schema.dump dvdrental

# One table
pg_dump -h pgsql01 -U backupuser -t public.customer -F c -f customer.dump dvdrental
```

Important: Use a `pg_dump` client version that is not older than the source server major version. Test restores when using logical dumps for migrations.

4. Native Tool: `pg_restore`

`pg_restore` restores non-plain `pg_dump` archives and can restore selectively or in parallel.

```
# Restore a custom-format archive
createdb -h pgsql02 -U postgres dvdrental_restore
pg_restore -h pgsql02 -U postgres -d dvdrental_restore /backup/dvdrental.dump

# Parallel restore
pg_restore -h pgsql02 -U postgres -j 8 -d dvdrental_restore /backup/dvdrental_dir

# Inspect archive contents before restore
pg_restore -l /backup/dvdrental.dump

# Restore only one table
pg_restore -h pgsql02 -U postgres -d dvdrental_restore -t public.customer /backup/dvdrental.dump
```

Security: A logical dump can contain SQL that creates objects and executes code during restore. Only restore trusted backups, and inspect dumps received from untrusted sources.

5. Native Tool: `pg_dumpall` and `psql`

`pg_dumpall` writes a cluster-wide SQL script and is especially useful for global objects that `pg_dump` does not include, such as roles and tablespaces.

```
# Global objects only
pg_dumpall -h pgsql01 -U postgres --globals-only > /backup/globals.sql

# All databases as SQL
pg_dumpall -h pgsql01 -U postgres > /backup/all_databases.sql

# Restore plain SQL
```

```
psql -h pgsq102 -U postgres -f /backup/globals.sql postgres
```

Enterprise pattern: A common logical safety net is per-database `pg_dump` plus `pg_dumpall --globals-only`. This complements, rather than replaces, physical backup and PITR.

6. Native Tool: `pg_basebackup`

`pg_basebackup` takes a physical base backup of a running PostgreSQL cluster. It can be used for PITR and as the starting point for a streaming-replication or log-shipping standby.

- Works at whole-cluster physical level; it is not a table-level backup tool.
- Uses the PostgreSQL replication protocol and normally requires a role with `REPLICATION` privilege or equivalent permissions.
- Supports plain and tar output formats.
- Can stream required WAL during the backup.
- Modern PostgreSQL supports full and incremental base backups.
- Native backup catalog/retention orchestration is limited compared with `pgBackRest` or `Barman`.

```
# Example full base backup
pg_basebackup -h pgsq101 -U replicator -D /backup/pg18_full -Fp -Xs -P

# Tar + compression example
pg_basebackup -h pgsq101 -U replicator -D /backup/pg18_tar -Ft -z -Xs -P
```

7. Native Incremental Backup and `pg_combinebackup`

PostgreSQL native incremental base backups reduce the amount of changed data copied after a prior backup. An incremental backup is not directly usable as PGDATA; it must first be combined with the backups on which it depends.

```
# Conceptual workflow
# 1) Take and retain a full base backup with its backup_manifest.
# 2) Take an incremental backup referencing a previous manifest.
# 3) Reconstruct a synthetic full backup before starting PostgreSQL.

pg_combinebackup /backup/full_001 /backup/incr_002 /backup/incr_003 -o /restore/synthetic_full
```

- Keep the full backup and every required incremental dependency until a new independent recovery chain is established.
- PostgreSQL does not automatically manage a full enterprise backup catalog or determine every backup dependency for retention.
- Use `pg_verifybackup` to validate the component backups; `pg_combinebackup` mainly validates that the backup chain relationships are correct.

8. Native Tool: `pg_verifybackup`

`pg_verifybackup` validates a `pg_basebackup` backup against the `backup_manifest` created by PostgreSQL. It can detect missing/extra files, checksum mismatches and several WAL-related problems.

```
# Verify a plain-format base backup
```

```
pg_verifybackup /backup/pg18_full

# Exit immediately on first error
pg_verifybackup --exit-on-error /backup/pg18_full
```

Do not stop at verification: PostgreSQL explicitly warns that `pg_verifybackup` cannot prove every condition required for a successful server startup. Enterprise standards should include real restore-and-recovery tests.

9. Native WAL Archiving and Point-in-Time Recovery (PITR)

PITR combines a physical base backup with the sequence of WAL records needed to replay changes from the backup checkpoint to a selected recovery target.

```
# postgresql.conf - simple example only
archive_mode = on
archive_command = 'test ! -f /pgarchive/%f && cp %p /pgarchive/%f'

# Recovery example
restore_command = 'cp /pgarchive/%f %p'
recovery_target_time = '2026-09-09 14:26:59-05:00'

# Create recovery signal in PGDATA before startup
touch $PGDATA/recovery.signal
```

Production caution: A simple `cp`-based `archive_command` is useful for labs, but enterprise designs normally add durability checks, monitoring, remote storage and retry behavior through a tested backup platform such as pgBackRest or Barman.

10. Native Low-Level Backup API

PostgreSQL provides low-level backup functions for specialized integration when `pg_basebackup` does not provide enough flexibility. These APIs require careful coordination of file copy and WAL retention.

```
SELECT pg_backup_start('enterprise_storage_backup');

-- Copy PGDATA and tablespaces with a storage/file-system mechanism.

SELECT * FROM pg_backup_stop();
```

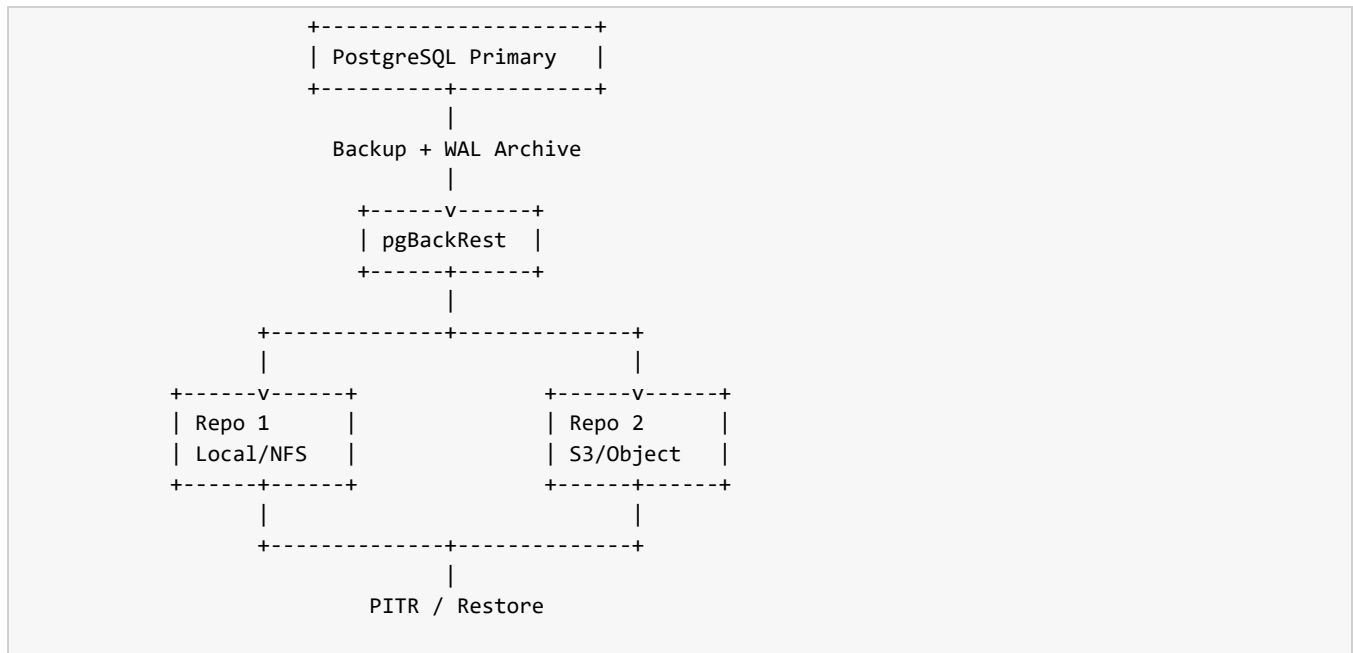
Use case: Use this only when integrating PostgreSQL with a controlled external backup/snapshot workflow. For normal DBA operations, prefer supported higher-level tooling.

11. Enterprise Tool: pgBackRest

pgBackRest is a purpose-built PostgreSQL backup and restore platform. It is a strong default choice for large self-managed production PostgreSQL environments.

Capability	pgBackRest
Backup types	Full, differential, incremental; block-level storage optimization available
Parallelism	Parallel backup and restore
WAL	archive-push/archive-get, asynchronous archiving
Repositories	Multiple repositories
Cloud/object storage	S3, Azure-compatible and GCS options documented by the project
Retention	Policy-driven expiration
Standby backup	Supported
Restore	PITR, delta restore, repository-based recovery
Integrity	Repository metadata/checksums and backup validation workflows
Best fit	Large enterprise PostgreSQL, local + remote repository, cloud/object storage, strong RPO/RTO

Reference architecture



Example configuration

```
# /etc/pgbackrest/pgbackrest.conf
[global]
repo1-path=/var/lib/pgbackrest
repo1-retention-full=2
repo1-retention-diff=4
start-fast=y
process-max=4

[pg18]
pg1-path=/var/lib/postgresql/18/main
```

```
# PostgreSQL WAL archive integration
archive_mode = on
archive_command = 'pgbackrest --stanza=pg18 archive-push %p'
restore_command = 'pgbackrest --stanza=pg18 archive-get %f "%p"'

# Create stanza and validate
pgbackrest --stanza=pg18 stanza-create
pgbackrest --stanza=pg18 check

# Full / differential / incremental
pgbackrest --stanza=pg18 --type=full backup
pgbackrest --stanza=pg18 --type=diff backup
pgbackrest --stanza=pg18 --type=incr backup

# View repository status
pgbackrest --stanza=pg18 info
```

Why pgBackRest is strong in enterprise

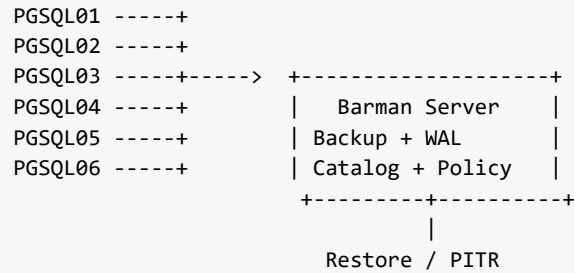
- Parallel backup and restore can reduce backup windows and RTO.
- Multiple repositories allow fast local recovery plus offsite/object-storage resilience.
- Differential and incremental strategies reduce repeated data transfer.
- Block-level repository storage can reduce space and transfer for changed large files.
- Integrated WAL archiving and PITR reduce the amount of custom scripting required.
- Retention and repository metadata reduce operational mistakes compared with manually managing native backup files.

12. Enterprise Tool: Barman

Barman (Backup and Recovery Manager) is an enterprise PostgreSQL backup platform maintained by EnterpriseDB. It is particularly strong when a dedicated backup server centrally manages multiple PostgreSQL servers.

Capability	Barman
Centralized management	Major strength: multiple PostgreSQL servers managed from a Barman server
Backup methods	PostgreSQL streaming/base-backup method, rsync-based methods, local-to-cloud and supported snapshot workflows
Incremental	File-level rsync incremental and block-level incremental capabilities depending on method/version
WAL	WAL archiving and WAL streaming using pg_receivewal
Retention	Redundancy or recovery-window policies
Cloud	barman-cloud tooling for object storage and supported cloud providers
Restore	PITR targets, get-wal, delta restore, remote recovery workflows
Best fit	Central backup server, multi-cluster estate, centralized catalog and policy control

Reference architecture



Example server configuration

```
# /etc/barman.d/pgsql01.conf
[pgsql01]
description = "Production PostgreSQL"
conninfo = host=pgsql01 user=barman dbname=postgres
streaming_conninfo = host=pgsql01 user=streaming_barman
backup_method = postgres
streaming_archiver = on
slot_name = barman
create_slot = auto
retention_policy = RECOVERY WINDOW OF 14 DAYS
```

```
# Validate and run
barman check postgresql01
barman backup postgresql01
barman list-backups postgresql01
```

```
# Recovery pattern
barman recover pgsql01 BACKUP_ID /restore/pgdata --target-time "2026-09-09 14:26:59-05:00"
```

Current-version note: Barman 3.20.0 was released on 27 August 2026. It requires Python 3.12 or later and includes modern cloud/WAL/restore capabilities. Always check the exact documentation for your deployed Barman release.

13. Cloud-Focused Tool: WAL-G

WAL-G is a backup and WAL archival tool designed around compressed/encrypted backups and remote/object storage. It is common in cloud-native and Kubernetes-oriented PostgreSQL architectures.

- Supports full and incremental/delta-style backup workflows.
- Pushes/fetches backups to remote storage without requiring a local long-term repository.
- Provides WAL push/fetch operations for PITR.
- Supports local filesystem backup and remote BASE_BACKUP protocol modes.
- Can run in containerized and Kubernetes environments.

```
# Conceptual WAL-G flow
wal-g backup-push $PGDATA
wal-g backup-list
wal-g backup-fetch /restore/pgdata LATEST

# PostgreSQL integration pattern
archive_command = 'wal-g wal-push %p'
```

```
restore_command = 'wal-g wal-fetch %f %p'
```

Best fit: Cloud-native environments where direct object-storage workflows are more important than a classic centralized backup server.

14. Other Third-Party Tools

Tool	Category	Strengths	Enterprise guidance
pg_probackup	Physical backup manager	Full/incremental, catalog, validation, merge/incremental restore features	Strong technical tool; evaluate project support model, platform fit and organizational standard before adoption.
pghoard	Backup/WAL service	Object storage, compression/encryption, WAL handling, monitoring-friendly daemon	Useful in existing Aiven/cloud-oriented estates; lower priority for new general DBA standardization than pgBackRest/Barman.
WAL-E	Legacy cloud WAL/backup tool	Historically important S3/WAL pattern	Treat mainly as legacy awareness; for new designs evaluate WAL-G or current enterprise tools.
pg_rman	Physical backup/recovery utility	Backup/recovery features in specific PostgreSQL ecosystems	Awareness level unless your organization already standardizes on it.

15. Storage Snapshots and Cloud-Managed Backups

Storage snapshots and managed-service backups can provide very fast recovery, but they are an infrastructure capability rather than a universal substitute for PostgreSQL-aware backup design.

- LVM/ZFS/SAN/cloud-volume snapshots must include all PostgreSQL data and tablespace volumes required for a consistent recovery.
- Configuration files outside PGDATA may require separate backup.
- Coordinate snapshots with PostgreSQL backup/WAL behavior when consistency across multiple volumes is required.
- For Amazon RDS PostgreSQL, automated backups and PITR create a new DB instance when restoring to a selected time.
- For managed PostgreSQL services, use the provider-native backup/PITR service as the primary platform capability and use logical exports for portability or object-level needs where appropriate.

16. Enterprise Comparison Matrix

Tool	Type	Backup style	Scope	PITR	Policy/catalog	Best fit	Verdict
pg_dump / pg_restore	Native	Logical	Object/db	No	Manual	Migration/object recovery	Essential complement, not sole DR
pg_basebackup	Native	Physical full/incr	Cluster	With WAL	Manual	Native physical backup	Strong foundation
pgBackRest	3rd party	Full/diff/incr	Cluster	Yes	Strong	Enterprise self-managed	Top general recommendation
Barman	3rd party	Physical + catalog	Multi-server	Yes	Strong	Centralized fleet backup	Top centralized recommendation
WAL-G	3rd party	Full/incr + WAL	Cluster	Yes	Medium	Cloud/object storage	Strong cloud-native option
pg_probackup	3rd party	Physical/incr	Cluster	Yes	Strong	Specialized estates	Evaluate support/standardization
pghoard	3rd party	Physical + WAL	Cluster	Yes	Medium	Cloud/object-store service	Existing-cloud estates
Snapshots	Infrastructure	Block/volume	Volume	With design	External	Fast recovery	Use as layer, not only layer

17. Architecture Patterns and Tool Selection

Pattern A - Small / simple PostgreSQL

```
pg_dump / pg_restore
+
pg_basebackup
+
WAL archive / PITR
+
Off-host copy
```

Good for small estates where operational simplicity matters and the DBA team can safely manage retention and recovery procedures.

Pattern B - Enterprise self-managed

```
PostgreSQL Primary/Standby
|
+---- pgBackRest ---- Local Repository
|                      Remote/Object Repository
|
+---- WAL Archive --> Both repositories
|
+--> PITR / DR
```

Recommended when you want a PostgreSQL-aware enterprise backup platform with strong backup types, parallelism, multiple repositories and recovery control.

Pattern C - Central backup service

```
PostgreSQL Server 01 --+
PostgreSQL Server 02 --+
PostgreSQL Server 03 ----> Barman Server --> Backup Catalog
PostgreSQL Server 04 --+                      WAL Archive
PostgreSQL Server 05 --+                      Cloud/Storage
```

Recommended when centralized policy, inventory, backup catalog and recovery orchestration across many PostgreSQL servers are key requirements.

Pattern D - Cloud native / Kubernetes

```

PostgreSQL Pod/VM
|
+--> WAL-G --> S3/GCS/Azure-compatible Object Storage
|
+--> Backup Metadata / PITR

```

Useful where direct object-storage integration and containerized operation are first-class design requirements.

18. RPO/RTO Design Guidance

Business requirement	Typical design direction	Notes
RPO 24 hours	Nightly logical or physical backup may be enough	Still test recovery and keep off-host copies
RPO 1 hour	Frequent backup + WAL archive	WAL continuity becomes critical
RPO minutes	Continuous WAL archiving/streaming + frequent validated base backups	Monitor archive lag and replication slot growth
RPO near zero	HA + continuous WAL + DR copy	Backup is still required; HA is not backup
Low RTO for multi-TB DB	Parallel restore, local repository, snapshots, standby-assisted backup	Prefer pgBackRest/Barman plus tested fast-storage recovery

Core rule: High Availability reduces outage time; backup protects against corruption, operator error, ransomware, logical mistakes and disasters. Do not treat replication as a backup replacement.

19. Backup Scheduling and Retention Examples

Example - pgBackRest

Schedule	Backup	Purpose
Sunday 01:00	Full	Independent weekly recovery base
Mon-Sat 01:00	Differential	Shorter backup window
Every 6 hours	Incremental	Reduce exposure and restore chain size
Continuous	WAL archive	PITR / RPO
Daily	Repository check + alert review	Operational assurance
Monthly	Full restore drill	Prove recoverability

Retention example: keep 2-4 full backups locally for fast recovery and a longer remote/object-storage recovery window according to compliance and business requirements.

20. Monitoring, Validation and Restore Testing

- Alert when WAL archiving stops or archive latency exceeds the RPO design.
- Alert when no successful backup exists inside the approved maximum age.
- Track backup duration, throughput, compression ratio, repository free space and backup growth.
- Track replication-slot retained WAL if a backup product uses streaming WAL.
- Validate backup metadata/checksums using the selected tool.
- Run automated restore tests into isolated infrastructure.
- Start PostgreSQL after restore, validate system identifier/version, connect to databases and execute application-level smoke tests.
- Record recovery time achieved versus RTO.

```
# Native examples
SELECT * FROM pg_stat_archiver;

SELECT pid, phase, backup_total, backup_streamed
FROM pg_stat_progress_basebackup;

# pgBackRest
pgbackrest --stanza=pg18 info
pgbackrest --stanza=pg18 check

# Barman
barman check pgsql01
barman list-backups pgsql01
```

21. Real-Time Failure and Recovery Scenarios

Accidental TRUNCATE at 14:27

If committed, restore a physical backup into an isolated target and replay WAL to a safe target immediately before the unwanted transaction. Validate recovered data, then perform the approved cutover/data-extract procedure.

Primary server destroyed

Provision replacement infrastructure, restore the latest valid physical backup, retrieve archived WAL, recover to the latest possible consistent point, validate, then redirect applications.

One table dropped

If a recent logical dump exists, restore the table selectively. Otherwise use PITR into an alternate cluster and extract the required table/data.

Backup repository unavailable

Fail over to an alternate repository or offsite copy if configured. pgBackRest multiple repositories are useful for this design.

Missing WAL segment

PITR cannot pass a required missing segment. Locate an alternate archive/replica copy or recover only to the last reachable consistent point. Investigate archiving failure before resuming service.

Corrupt backup file

Reject the backup, verify adjacent backup sets, use the next valid chain, and conduct an incident review. This is why verification plus restore testing is required.

Ransomware on DB host

Restore from immutable/off-host repository or provider backup. Rotate credentials and validate the recovery environment before reconnecting production.

Multi-TB recovery exceeds RTO

Use parallel restore, local high-throughput repository, snapshot-assisted strategy, pre-provisioned DR capacity or a continuously maintained standby in addition to backups.

22. Security and Compliance Controls

- Use least-privilege backup roles. Replication permissions should be granted only where required.
- Do not embed production passwords in shell history or scripts. Use .pgpass, secret managers or protected service credentials.
- Encrypt backups in transit and at rest where required.
- Separate backup-admin permissions from database-admin permissions for higher assurance environments.
- Use immutable/object-lock/WORM controls when ransomware or compliance risk justifies them.
- Protect encryption keys separately from encrypted backups.
- Audit backup deletion and retention changes.
- Restrict restore infrastructure because backup data contains production information.

23. Production Runbook Checklists

Daily checklist

- Latest backup successful and within expected duration.
- WAL archiving healthy; no unexpected failed_count growth.
- Repository/cloud storage reachable.
- Free space and object-storage growth within threshold.
- No stale replication slot retaining excessive WAL.
- Backup alerts and job failures reviewed.

Monthly restore drill

- Select a representative backup set without using the production server.
- Provision isolated recovery host/cluster.
- Restore backup and required WAL.
- Recover to latest or a specified PITR target.
- Start PostgreSQL and verify logs.
- Validate critical databases, tables, row counts and application smoke queries.
- Record RTO achieved and any manual dependencies.
- Destroy test environment securely after evidence is captured.

Before changing retention

- Confirm recovery-window requirements.
- Confirm dependency chains for incremental backups.
- Confirm legal/compliance retention.
- Confirm offsite/immutable copies.
- Test deletion in non-production repository if the change is complex.

24. Interview / Scenario Questions

Q: Why is `pg_dump` not enough for enterprise DR?

A: It is logical and excellent for portability/object-level recovery, but it does not by itself provide continuous WAL-based PITR, fast whole-cluster physical recovery, or enterprise retention/repository management.

Q: What is the biggest operational advantage of `pgBackRest`?

A: Integrated PostgreSQL-aware backup types, parallelism, WAL/PITR, retention and multiple repositories reduce custom scripting and improve recovery operations.

Q: When would you choose Barman over `pgBackRest`?

A: When a dedicated centralized backup server and centralized catalog/policy across many PostgreSQL servers is a major architecture requirement.

Q: What changed with native PostgreSQL incremental backup?

A: Modern PostgreSQL can create incremental base backups, and `pg_combinebackup` reconstructs a synthetic full backup. The DBA must still manage dependency chains and retention carefully.

Q: Does `pg_verifybackup` guarantee a backup will restore?

A: No. It verifies many integrity conditions against the manifest, but PostgreSQL documentation still recommends actual restore testing.

Q: Why must WAL be protected?

A: A base backup can only recover forward to a target if all required WAL records are available.

Q: Is streaming replication a backup?

A: No. Logical mistakes and corruption can replicate. HA and backup solve different risks.

Q: What is the difference between RPO and RTO?

A: RPO is acceptable data loss; RTO is acceptable recovery duration.

Q: When is WAL-G attractive?

A: Cloud-native/object-storage environments, including container/Kubernetes deployments, where direct remote backup/WAL workflows are desired.

Q: Why keep two repositories?

A: To reduce a single failure domain: for example, local high-speed restore plus remote/offsite disaster recovery.

25. Final Recommendations

Environment	Primary recommendation	Secondary/complementary
Small / lab	<code>pg_basebackup</code> + WAL + <code>pg_dump</code>	<code>pg_verifybackup</code> + restore drill

General enterprise self-managed	pgBackRest	pg_dump/pg_restore for logical recovery; snapshots where justified
Large centralized multi-server estate	Barman	Logical exports; object storage/cloud integration
Cloud-native/Kubernetes	WAL-G or operator-integrated backup	Provider snapshots and logical exports
Managed PostgreSQL service	Provider-native backup/PITR	pg_dump/pg_restore for portability/object recovery
Multi-TB low-RTO system	pgBackRest/Barman + local fast repository + WAL	Snapshot/DR standby + remote immutable copy

Recommended learning order: 1) pg_dump/pg_restore/pg_dumpall, 2) pg_basebackup, WAL and PITR, 3) pg_verifybackup and pg_combinebackup, 4) pgBackRest, 5) Barman, 6) WAL-G, 7) snapshots and cloud-managed backup.

There is no authoritative worldwide usage census proving one PostgreSQL backup product is the single 'most popular'. The enterprise recommendation in this SOP is based on capability, operational maturity, architecture fit and current official documentation rather than an unverified popularity claim.

26. Official References

PostgreSQL 18 - Backup and Restore

<https://www.postgresql.org/docs/current/backup.html>

PostgreSQL 18 - Continuous Archiving and PITR

<https://www.postgresql.org/docs/current/continuous-archiving.html>

PostgreSQL 18 - pg_dump

<https://www.postgresql.org/docs/current/app-pgdump.html>

PostgreSQL 18 - pg_dumpall

<https://www.postgresql.org/docs/current/app-pg-dumpall.html>

PostgreSQL 18 - pg_restore

<https://www.postgresql.org/docs/current/app-pgrestore.html>

PostgreSQL 18 - pg_basebackup

<https://www.postgresql.org/docs/current/app-pgbasebackup.html>

PostgreSQL 18 - pg_combinebackup

<https://www.postgresql.org/docs/current/app-pgcombinebackup.html>

PostgreSQL 18 - pg_verifybackup

<https://www.postgresql.org/docs/current/app-pgverifybackup.html>

pgBackRest - User Guide

<https://pgbackrest.org/user-guide.html>

Barman 3.20.0 - Documentation

<https://docs.pgbarman.org/release/3.20.0/>

Barman 3.20.0 - Cloud

https://docs.pgbarman.org/release/3.20.0/user_guide/barman_cloud.html

WAL-G - PostgreSQL Documentation

<https://github.com/wal-g/wal-g/blob/master/docs/PostgreSQL.md>

pg_probackup

https://github.com/postgrespro/pg_probackup

pghoard

<https://github.com/Aiven-Open/pghoard>

Amazon RDS - Point-in-time restore

https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_PIT.html